CS 300 – Chapter 5 Lectures

Hash Tables and Cryptographic Hashing

Jose de Lima

April 23, 2026

Key idea Chapter map

Where this sits in CS 300. Until now, lookup costs $O(n)$ (linear search) or $O(\log n)$ (binary search, balanced trees). Chapter 5 introduces **hashing**, the technique that takes lookup *on average* down to $O(1)$. The catch is everywhere: bad hash functions, full tables, adversarial inputs, and resizing. Mastering hash tables is mastering the tradeoffs that make $O(1)$ possible.

What you add to your toolkit here:

- **Hash function** as a deterministic $key \to index$ map, with the requirements (fast, well-distributed, deterministic).
- **Collision resolution**: chaining (buckets are linked lists) vs. open addressing (linear / quadratic probing, double hashing).
- **Load factor** $\alpha = n/\text{capacity}$: the single number that predicts whether your table is fast or sad.
- **Resizing** (rehashing) as the mechanism that keeps amortized $O(1)$ honest.
- **Direct hashing** – the degenerate case where keys *are* indices. Works when the key space is dense and small.
- **Cryptographic hashing** – same mathematical object, entirely different requirements (collision resistance, preimage resistance, slowness). Don't confuse the two.

Mastery checklist – you should be able to, without references:

1. Define a hash function and state the three desiderata (fast, deterministic, uniform).
2. Given a key stream, draw the resulting table under chaining AND under linear probing, starting from capacity 7.
3. State the expected cost of insert/find/remove under chaining and under open addressing, and say what happens as α approaches 1.
4. Compute the mid-square or multiplication-method hash of a concrete key by hand.
5. Explain *primary clustering* (linear probing) and *secondary clustering* (quadratic probing) and how double hashing escapes both.
6. Describe the rehash procedure: when it triggers, what the new capacity is, and why you can't just copy the old table – you must rehash every element.
7. Say why `std::unordered_map` uses chaining (not open addressing) and what that buys vs. costs.
8. Distinguish cryptographic and data-structure hashing, and name one attack (e.g., hash flooding) that bridges them.

Looking ahead. Hash tables will show up in graph algorithms as "visited" sets (opt. ch. 10), in DP as memoization tables, and as the default *map* container in every production C++ codebase

(`std::unordered_map`). Chapter 6 (BSTs) gives you the *ordered* alternative – slower on average, but with guarantees hash tables lack.

1 5.1 Hash Tables: The Big Idea

Every data structure you've seen so far has been one of two shapes: a *list* (search costs $O(n)$) or a *sorted array* (search costs $O(\log n)$). A hash table does something radically different. Instead of searching for a key, it *computes* where the key should live. Search, insert, and remove become $O(1)$ on average — constant time, regardless of how many items are stored.

Definition — Hash table

A data structure that stores items in an array and uses a *hash function* to compute, from each item's *key*, the array index ("bucket") where the item should be stored. Lookup, insertion, and removal are therefore one arithmetic computation plus one array access — $O(1)$ on average.

Aside Hash tables implement the Set ADT

Recall the Sequence-vs-Set framing from §4.1: every container belongs to one of two interface families. **Sequence** keeps items at caller-chosen positions ($i = 0, 1, \ldots, n-1$); **Set** keeps items at positions chosen by the data structure itself, looked up by an *intrinsic key*. ch. 4 built Sequence implementations (lists, stacks, queues, deques). *This chapter and ch. 6 build Set implementations.*
A hash table is a Set: the operations are `find(k)`, `insert(x)`, `delete(k)` — never "insert at position i" or "the third item." In C++:

- `std::unordered_set<K>` — the bare Set: "is key k present?"
- `std::unordered_map<K,V>` — a *dictionary* (Set without the ordered ops, in OCW terminology): "what value is associated with key k?"

ch. 5 covers the chained and open-addressed implementations of the Set interface. §5.8 covers the special case where keys *are* small integers and the hash function is the identity — the trivial Set implementation. ch. 6 covers BSTs, the alternative Set implementation that gives up O(1) lookup in exchange for ordered operations (`find_min`, `find_next(k)`).

The core trick

Suppose you have an array of size 10 and you want to store the key 25. Where does it go?

```
index = 25 % 10 = 5
```

Store it at `array[5]`. To look it up later: compute the same index, read that slot. No search. No comparison loop. The work of "finding" has been replaced by a calculation that locates the element directly.

Example Simplest possible hash table

```
constexpr size_t N = 10;
int table[N];

void insert(int key) { table[key % N] = key; }
bool contains(int key) { return table[key % N] == key; }
```

Three lines, $O(1)$ in both operations. This is the entire idea. Everything else in this chapter — hash functions, collisions, chaining, probing, load factor — is about making this trick *actually work* in practice when keys don't divide up so neatly.

Why not just use a giant array?

If keys are small integers, you already can. An array of size 2^{32} indexed directly by `int` keys works — but that's 16 GB of memory to hold possibly a handful of values. Hash tables compress a large *potential* key space into a small array, trading perfect placement for practical memory use.

Approach	Lookup	Space
Direct array (key = index)	$O(1)$	$O(K)$ where K = key range
Sorted array + binary search	$O(\log n)$	$O(n)$
Linked list	$O(n)$	$O(n)$
Hash table	$O(1)^*$	$O(n)$

* average case; worst case can degrade, see later sections.

The hash table gets you array-speed lookup at list-space cost. That combination is so valuable that hash tables are the backbone of almost every real-world fast key-value store: JavaScript objects, Python dicts, Java `HashMap`, Redis, DNS caches, database indexes (sometimes), compiler symbol tables, the list goes on.

Vocabulary

Term	Meaning
Key	The value used to look up an item
Value	The data associated with the key (often absent — just a set)
Bucket	A slot in the underlying array
Hash function	`key` $\rightarrow$ `bucket index`
Collision	Two keys that hash to the same bucket
Load factor	n items / m buckets — a density measure

Key idea Collisions are the entire problem

The naive hash table above breaks the instant two keys hash to the same bucket. With 10 buckets and key `% 10`, keys 25 and 75 both land at index 5 — the second insert either overwrites the first or needs a plan B. The rest of the chapter is "plan B": different strategies for resolving collisions. The big split is *chaining* (each bucket holds a list of items) vs. *open addressing* (find another empty bucket in the array). Each has different cost/space tradeoffs.

Gotcha $O(1)$ is the *average* case

A hash table's $O(1)$ lookup is the average case under a good hash function and reasonable load. Worst case, every key hashes to the same bucket and lookup degrades to $O(n)$ — the same as a linked list. Two defenses: pick a good hash function (distribution matters), and keep the load factor low (resize the array when it fills up). Sections 5.5–5.8 make this concrete.

Aside Hash tables vs. STL

C++ gives you two hash-based containers in `<unordered_map>` and `<unordered_set>`. Both are chaining hash tables. The older `std::map` / `std::set` are *balanced BSTs* (red-black trees, chapter 6), which is why their operations are $O(\log n)$ instead of $O(1)$. "Ordered" in the name is the giveaway: iterating a `map` gives sorted order; iterating an `unordered_map` gives whatever order the hash function happened to produce. If you don't need ordering, unordered is usually faster.

Key idea Chapter 5 roadmap

This chapter is structured as: hash table idea → hash functions → collision resolution (two strategies) → cost analysis and load factor. By the end, you'll be able to implement one from scratch and reason about its performance. The key insight to hold onto: *a hash table is an array accessed through a function, not through a search loop.* Everything else is engineering around that idea.

2 5.2 Chaining

Chaining is the first collision-resolution strategy. The idea is almost embarrassingly simple: if two keys want the same bucket, don't pick a winner — let both live there, in a list.

Definition — Chaining

A collision-resolution strategy in which each bucket stores a *list* of items rather than a single item. On insertion, the new item is appended to the bucket's list; on lookup, the bucket's list is searched for the matching key.

The mental model

Picture the hash table as an array of *list heads* instead of an array of values. Each slot is either `nullptr` (empty) or points to a chain of nodes that all hashed to that index.

Bucket	Chain
0	`nullptr`
1	21 → 31 → 51
2	`nullptr`
3	13
4	24 → 74

Every chain only contains keys that hashed to that index, so the search step only compares against "nearby" items — ideally just one or two, not the entire table.

Algorithms

Each of the three operations becomes: (1) hash the key to find the bucket, (2) do the list-level operation on that bucket's chain.

Example Chained hash table operations

```
template <typename T>
class ChainedHashTable {
```

```
    struct Node { T data; Node* next; };
    std::vector<Node*> table;
    size_t n_ = 0;

    size_t hash(const Key& k) const {
        return std::hash<Key>{}(k) % table.size();
    }

public:
    explicit ChainedHashTable(size_t m) : table(m, nullptr) {}

    bool contains(const Key& k) const {
        for (Node* p = table[hash(k)]; p; p = p->next)
            if (p->data.key == k) return true;
        return false;
    }

    void insert(const T& item) {
        size_t b = hash(item.key);
        for (Node* p = table[b]; p; p = p->next)
            if (p->data.key == item.key) { p->data = item; return; }
        table[b] = new Node{item, table[b]};  // prepend
        ++n_;
    }

    bool remove(const Key& k) {
        size_t b = hash(k);
        Node** cur = &table[b];
        while (*cur) {
            if ((*cur)->data.key == k) {
                Node* dead = *cur;
                *cur = dead->next;
                delete dead;
                --n_;
                return true;
            }
            cur = &(*cur)->next;
        }
        return false;
    }
};
```

Two implementation notes: `insert` checks for duplicates first because most hash-table APIs are "unique-key" (insert acts as upsert), and `remove` uses the `Node**` trick from chapter 4 so the same code handles head and interior removal without a special case.

Cost

Operation	Average	Worst	Notes
`insert`	$O(1+\alpha)$	$O(n)$	Chain walk to check duplicate
`contains`	$O(1+\alpha)$	$O(n)$	Chain walk to compare keys
`remove`	$O(1+\alpha)$	$O(n)$	Chain walk to locate node

$\alpha = n/m$ = *load factor* = average chain length. When α is a small constant (say, ≤ 1), every operation is effectively $O(1)$. When α grows, operations slow linearly. Section 5.6 discusses how to keep α bounded by resizing.

Key idea The key insight

With a good hash function and a reasonably-sized table, chains stay short — often length 0, 1, or 2. So even though the *worst* case is $O(n)$, the *average* case is a handful of comparisons. The $O(1)$ claim for hash tables is really "$O(1+\alpha)$ where α is kept small by good engineering."

Pros and cons

Pro	Never fills up — you can keep inserting past the table size, chains just grow longer.
Pro	Removal is easy — just unsplice a list node.
Pro	High load factors tolerable (still works at $\alpha > 1$).
Con	Extra memory per item (one `next` pointer per node).
Con	Cache-unfriendly — chain walks follow pointers all over the heap, same as plain linked lists.
Con	Extra allocation on every insert (one `new Node`).

Gotcha A degenerate hash function degrades to a list

If every key hashes to the same bucket, chaining leaves you with a single linked list of length n — and every operation is $O(n)$. This is not hypothetical: some attackers exploit predictable hash functions to deliberately trigger this ("hash-collision DoS"). Modern libraries use randomized hash seeds to make collision attacks impractical. If you roll your own hash function, be aware that "obvious" functions (like `key % m` with adversarial keys) can produce this worst-case behavior.

Aside Why $O(1+\alpha)$? The simple uniform hashing assumption

The cost claim "find takes $O(1+\alpha)$ on average" isn't a slogan; it's a theorem (CLRS Ch 11.2, Theorems 11.1–11.2). The proof rests on one assumption:

Simple uniform hashing. Any given key is equally likely to hash to any of the m slots, independently of where any other key has hashed.

Under this assumption, the expected length of bucket $T[h(k)]$ is $E[n_{h(k)}] = n/m = \alpha$. So an unsuccessful search examines α elements on average, plus $O(1)$ to compute the hash and index — total $\Theta(1+\alpha)$. A symmetric argument (with slightly more bookkeeping for the "which list is searched" probability) gives the same $\Theta(1+\alpha)$ bound for successful search.

Two consequences worth internalizing:

- If you keep α bounded by a constant (which is what resizing in §5.6 does), the $1+\alpha$ term is $O(1)$. *This is the entire reason hash tables average $O(1)$.*

- The open-addressing variants in §5.3–§5.5 all use the same simple-uniform-hashing assumption. Their per-strategy cost formulas (linear's $\frac{1}{2}(1 + 1/(1 - \alpha))$, double-hashing's tighter bounds) come from analyzing different probe sequences *under that same assumption.*

The assumption is strong — real hash functions don't perfectly satisfy it on all inputs. §5.7 shows what hash functions try to do to approximate it; §5.7 (universal hashing subsection) shows how to make a probabilistic guarantee without the assumption at all.

Aside What the STL does

`std::unordered_map` and `std::unordered_set` are *required* by the standard to use chaining (separate chaining, in formal terminology). This is a correctness requirement, not just an implementation choice: the standard guarantees that inserting doesn't invalidate iterators to other elements, which essentially forces per-bucket lists. The alternative, open addressing (next section), moves elements around on insertion and would break iterator stability.

Key idea The contrast with open addressing

Chaining solves collisions by *going outside the array* — add extra memory in the form of chains. The next section covers the opposite approach: *stay inside the array* and find a different bucket for the collider. Both get $O(1)$ average, with different space/cache tradeoffs. Know both; the choice depends on context.

3 5.3 Linear Probing

Linear probing is the first flavor of *open addressing* — the "stay inside the array" approach to collisions. When the computed bucket is occupied, walk forward through the array one slot at a time until you find an empty one. That's it. Simple to describe, surprisingly subtle to get right.

Definition — Linear probing

A collision-resolution strategy that stores all items directly in the hash table array. On collision, it checks bucket $h + 1$, then $h + 2$, then $h + 3$, …(wrapping around the end), until an empty bucket is found. The sequence $h, h + 1, h + 2, \ldots$ is called the *probe sequence.*

Why it's interesting

Chaining (5.2) allocates a new node per item. Linear probing does *not* — every item lives in the fixed array itself.

Aspect	Chaining	Linear probing
Extra memory per item	1 pointer	0
Allocations on insert	1 `new Node`	0
Cache behavior	Poor (heap jumps)	Excellent (contiguous)
Can overflow?	Never	Yes (array full ⇒ insert fails)
Removal	Unsplice list node	Tombstone (see below)

For small-to-medium tables, linear probing typically *wins* on real hardware: no allocation, no pointer chasing, and the CPU prefetches the next probe slot because it's literally the next cache line. This is why many high-performance hash tables (Google's `dense_hash_map`, Robin Hood hashing, Swiss tables) are built

on open addressing rather than chaining.

Insert

Example Linear-probe insert

```
bool insert(int key) {
    size_t start = hash(key) % N;
    for (size_t i = 0; i < N; ++i) {
        size_t b = (start + i) % N;
        if (table[b].state != OCCUPIED) {
            table[b] = { key, OCCUPIED };
            return true;
        }
        if (table[b].key == key) return false; // duplicate
    }
    return false;  // table full
}
```

The `% N` wrap makes the probe sequence circular — if you hit the end of the array, you keep going from index 0.

The tombstone problem

Removal on an open-addressed table is *not* just "mark the bucket empty." Consider:

1. Insert 25 (bucket 5).
2. Insert 35 (bucket 5 occupied, probe to 6).
3. Remove 25 — mark bucket 5 empty.
4. Search for 35 — check bucket 5, find it empty, stop searching. Returns "not found." **But 35 is there**, in bucket 6.

To fix this, open-addressed tables distinguish *two* kinds of empty:

State	Meaning
`EMPTY_SINCE_START`	Never held an item; search can stop here
`OCCUPIED`	Currently holds an item
`EMPTY_AFTER_REMOVAL` (tombstone)	Was occupied; search continues past

The tombstone preserves the "I was here" information that the probe sequence needs. Insertions treat tombstones as empty (can reuse the slot). Searches treat tombstones as occupied (keep probing past).

Gotcha Tombstones never clean themselves up

A long-lived table with many insertions and removals can fill up with tombstones, slowing searches to a crawl. The fix is to *rehash*: allocate a fresh table, re-insert every live item, throw the old table away. The tombstones are gone, probes are short again. Sections 5.6–5.8 discuss when and how to trigger this.

Search

Example Linear-probe search

```
int* find(int key) {
    size_t start = hash(key) % N;
    for (size_t i = 0; i < N; ++i) {
        size_t b = (start + i) % N;
        if (table[b].state == EMPTY_SINCE_START) return nullptr;
        if (table[b].state == OCCUPIED && table[b].key == key)
            return &table[b].key;
        // tombstone: keep looking
    }
    return nullptr;
}
```

The three termination conditions matter: (1) found a match, return it; (2) hit a never-occupied slot, definitely not present; (3) probed every bucket, not present.

Cost and load factor

Linear probing's cost depends dramatically on how full the table is. Let $\alpha = n/m$ (load factor).

Load factor α	Avg probes (successful)	Feel
0.5	≈ 1.5	Fast
0.75	≈ 2.5	Still OK
0.9	≈ 5.5	Getting sluggish
0.95	≈ 10.5	Noticeable
0.99	≈ 50	Disaster

These numbers aren't linear in α — they grow like $\frac{1}{1-\alpha}$ (Knuth's analysis). That's why open- addressed tables typically enforce a much lower max load factor than chained ones: 0.5–0.75 is common for linear probing versus 1.0+ for chaining.

Key idea Primary clustering

Linear probing has a pathological behavior called *primary clustering*: once a run of occupied slots forms, any key that hashes *into* that run will extend it, making it more likely to catch future keys too. Clusters grow like snowballs. This is why quadratic probing (5.4) and double hashing (5.5) exist — they break the cluster-growth pattern by making probe sequences depend on the key, not just the starting bucket.

Aside Why linear probing is still worth using

Despite primary clustering, linear probing often beats "smarter" probe sequences in practice because of cache behavior. Every probe is the next cache line — no pointer chasing, no random access. Quadratic probing and double hashing have theoretically-shorter probe sequences but jump around the array, busting the cache. At load factors under 0.75, linear probing's wall-clock speed is hard to beat.

Key idea Key takeaways

Open addressing trades "cheap removal" for "no allocation" and "great cache behavior." The tombstone mechanism is the essential correctness fix. Load factor discipline is the essential performance fix — probe counts explode near $\alpha = 1$. The next sections show variants of probing that keep the same flavor but sidestep primary clustering.

4 5.4 Quadratic Probing

Linear probing's flaw, remember, is *primary clustering*: once a run of occupied buckets forms, any key that hashes into that run lengthens it, snowballing the problem. *Quadratic probing* sidesteps clustering by using a probe sequence whose gaps grow with each step.

Definition — Quadratic probing

An open-addressing collision-resolution strategy where the probe sequence for a key k is

$$\text{bucket}_i = (h(k) + c_1 i + c_2 i^2) \bmod m$$

for $i = 0, 1, 2, \ldots$ and chosen constants c_1, c_2. The quadratic term makes successive probes spread out instead of walking contiguously.

The probe pattern

With $c_1 = 0, c_2 = 1$ (the simplest common choice), the probe sequence from a starting bucket h visits:

i	Offset i^2	Bucket
0	0	h
1	1	$h + 1$
2	4	$h + 4$
3	9	$h + 9$
4	16	$h + 16$

The gaps grow: 1, 3, 5, 7, 9, …The probe sequence jumps past nearby occupied slots quickly, so even if there's a cluster starting near h, it gets skipped over rather than extended.

Example Quadratic-probe insert

```
bool insert(int key) {
    size_t h = hash(key);
    for (size_t i = 0; i < N; ++i) {
        size_t b = (h + c1 * i + c2 * i * i) % N;
        if (table[b].state != OCCUPIED) {
            table[b] = { key, OCCUPIED };
            return true;
        }
        if (table[b].key == key) return false; // duplicate
    }
    return false;  // full or unreachable
}
```

Everything else (tombstones, search, remove) carries over from linear probing. The only change is the probe-step formula.

Primary clustering: solved. Secondary clustering: new problem

Quadratic probing eliminates primary clustering (adjacent keys no longer share probe paths). But two keys that hash to the same *starting* bucket will still follow the exact same probe sequence — that's *secondary clustering*. Less severe than primary, but not gone.

Strategy	Primary clustering	Secondary clustering
Linear probing	Yes	Yes
Quadratic probing	No	Yes
Double hashing (5.5)	No	No

Gotcha Quadratic probing may not visit every bucket

With a poorly-chosen N and constants c_1, c_2, the probe sequence can cycle through a small set of buckets forever and miss the rest of the table — insert fails even though empty buckets exist. There's a theorem: if N is prime and the load factor is less than 0.5, the probe sequence *is* guaranteed to visit $N/2$ distinct buckets. So: pick prime N, keep $\alpha \leq 0.5$. If you need higher load factors, use linear probing or double hashing instead.

Choosing constants

$c_1 = 0, c_2 = 1$ gives you $h + i^2$, which has the guarantee above. $c_1 = 1, c_2 = 1$ gives $h + i + i^2$, which is the form used in the textbook pseudocode; it still works but the theoretical guarantees depend on specific N values. When in doubt, prefer the simpler $h + i^2$ version with prime table sizes.

Example Triangular-number probing

A well-known trick uses $c_1 = c_2 = 1/2$, giving probes at offsets $\frac{i(i+1)}{2}$ (the triangular numbers: 0, 1, 3, 6, 10, 15, …). This form visits *every* bucket when N is a power of 2 — a major convenience because hash tables often use power-of-2 sizes for fast modulo via bit masking. This is the form used in Java's older hash tables and many game-engine hash maps.

Cost

Load factor α	Avg probes (successful)	Notes
0.5	≈ 1.4	Slightly better than linear at 0.5
0.75	≈ 2.0	Noticeably better
0.9	≈ 3.2	Much better than linear's 5.5

At high load factors, quadratic probing substantially outperforms linear probing. At low load factors, the difference is minor and linear's cache-friendliness can win on wall-clock time.

Key idea Quadratic vs linear: the real tradeoff

Linear probing: simple, cache-friendly, suffers at high load. Quadratic probing: spreads clusters, works better at moderate load, slightly less cache-friendly (the jumps in the probe sequence skip cache lines). In modern hash table implementations that push load factors close to 1, the cache penalty is worth paying to avoid primary clustering. For textbook problems and moderate loads, either is fine.

Aside Nobody in production uses "pure" quadratic probing

Real-world implementations combine ideas: Google's Abseil Swiss tables use linear probing with SSE-accelerated parallel comparison; Rust's `HashMap` uses Robin Hood hashing (a linear-probing variant that equalizes probe lengths); CPython's `dict` uses a perturbation scheme that's neither pure linear nor pure quadratic. The textbook probes are pedagogy; the real insight is "open addressing plus anything that breaks clustering."

5 5.5 Double Hashing

Linear and quadratic probing both suffer *secondary clustering*: any two keys that hash to the same starting bucket follow the same probe sequence. *Double hashing* fixes this by making the probe *step size* also depend on the key — so two keys starting at the same bucket diverge immediately.

Definition — Double hashing

An open-addressing strategy that uses two hash functions. The probe sequence for a key k is

$$\text{bucket}_i = (h_1(k) + i \cdot h_2(k)) \bmod m$$

for $i = 0, 1, 2, \ldots$. h_1 determines the starting bucket; h_2 determines the step size between probes.

The idea, geometrically

Think of each key as having its own "rhythm" for walking through the table. Linear probing: everyone steps by 1. Quadratic: everyone's step pattern is $1, 4, 9, \ldots$. *Double hashing: each key's step depends on the key itself.*

Two keys can collide on their starting bucket and *still* separate instantly because their h_2 values differ. That's the whole trick.

Example

Let $m = 11$, $h_1(k) = k \bmod 11$, $h_2(k) = 5 - (k \bmod 5)$.

Key k	$h_1(k)$	$h_2(k)$	**Probe sequence**
55	0	5	0, 5, 10, 4, 9, 3, …
66	0	4	0, 4, 8, 1, 5, …

Both keys start at bucket 0, but after the first collision they walk in entirely different directions. That's why double hashing approximates "uniform hashing" — the theoretical ideal where every key's probe sequence is a random permutation of buckets.

Implementation

Example Double-hashing insert

```
bool insert(int key) {
    size_t h1 = hash1(key) % N;
    size_t h2 = hash2(key);   // must be nonzero, coprime to N
    for (size_t i = 0; i < N; ++i) {
```

```
        size_t b = (h1 + i * h2) % N;
        if (table[b].state != OCCUPIED) {
            table[b] = { key, OCCUPIED };
            return true;
        }
        if (table[b].key == key) return false; // duplicate
    }
    return false;  // table full
}
```

Search and remove use the same probe formula with the usual tombstone rules. The only new concern is *choosing h_2 correctly.*

The h_2 requirement: never zero, always coprime to m

If $h_2(k) = 0$, the probe sequence stays stuck at $h_1(k)$ forever — infinite loop. Always enforce $h_2(k) \neq 0$.

If $h_2(k)$ shares a factor with m (table size), the probe sequence visits only a subset of buckets — you can fail to insert even with empty buckets available. The standard fix: **make m a prime and design h_2 to always return a value in** $[1, m-1]$. A common construction:

$$h_2(k) = p - (k \bmod p), \text{ where } p < m \text{ is prime}$$

This guarantees $1 \leq h_2(k) \leq p < m$, and since m is prime, $\gcd(h_2(k), m) = 1$ always.

Gotcha Powers of 2 are incompatible with double hashing

If m is a power of 2, the only values coprime to m are the odd ones. So h_2 has to always return an odd number — often enforced by $h_2(k) = 2 \cdot g(k) + 1$. This works but limits h_2's range. Most double-hashing implementations use prime table sizes for this reason, even though modulo-by-prime is slower than modulo-by-power-of-2.

Cost

Load factor α	Avg probes (successful)	Notes
0.5	≈ 1.4	Matches quadratic
0.75	≈ 1.8	Better than quadratic
0.9	≈ 2.6	Significantly better
0.95	≈ 3.2	Holds up where others collapse

The cost formula is $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$ – essentially the theoretical best achievable by any open- addressing scheme. Double hashing's probe distribution is statistically close to uniform random, which is what the analysis assumes.

Strategy	Primary clusters?	Secondary clusters?	Cache-friendly?
Chaining	—	—	No
Linear probing	Yes	Yes	Yes
Quadratic probing	No	Yes	Somewhat
Double hashing	No	No	No

Key idea Why double hashing isn't always used

Double hashing has the best theoretical probe counts, yet most modern hash table implementations use linear probing. The reason is cache. Double hashing's jumps across the table are random accesses from the CPU's point of view; linear probing's contiguous walk feeds the cache. At load factors below 0.8, linear probing's cache advantage often beats double hashing's shorter probe count in wall-clock time. Above 0.8, double hashing wins. The lesson: big-O is not the whole cost story.

Aside Two hash functions from one

A common trick when you only have one hash function: split its output. For a 64-bit hash $H(k)$, use the low 32 bits as h_1 and the high 32 bits as h_2. This gives you two statistically-independent-enough hash values from one computation. Java's `HashMap` uses a variant of this idea.

Key idea The pattern across 5.2–5.5

Four collision strategies, one theme: *spread the keys out so probe paths are short.* Chaining spreads into separate lists. Linear probing walks, and clusters. Quadratic widens the walk. Double hashing randomizes the walk. Each fixes a specific flaw of the previous. Real-world hashing takes the best pieces of each: open-addressed layout (for cache), randomized probing (for spread), low load factors (for speed), and periodic rehashing (for cleanup). The remaining sections tie these together.

6 5.6 Hash Table Resizing

Every cost analysis so far has assumed the load factor stays bounded. But the user keeps inserting, and n keeps growing. Without intervention, $\alpha = n/m$ climbs past 0.5, 0.75, 0.9, and eventually the table fills up. *Resizing* is how hash tables bound their load factor at runtime.

Definition — Resize (rehash)

The operation of allocating a larger array, reinserting every existing item into it using a fresh hash-to-bucket mapping, and discarding the old array. Triggered when the load factor crosses a threshold. Cost: $O(n)$ — every item must be rehashed.

The algorithm

1. Decide a new size m'. Commonly $m' \approx 2m$, rounded up to the next prime.
2. Allocate a fresh array of size m', all slots empty.
3. For each non-empty bucket in the old table, re-compute $h(key) \bmod m'$ (note: *not* $\bmod m$) and insert into the new table.
4. Free the old array. Swap the pointer.

Example Resize

```
void resize(size_t newCap) {
    std::vector<Slot> oldTable = std::move(table);
    table.assign(newCap, Slot{});   // all EMPTY_SINCE_START
```

```
    n_ = 0;
    for (const Slot& s : oldTable) {
        if (s.state == OCCUPIED) insert(s.key);
    }
}
```

The old bucket indices are *meaningless* for the new table. Every item must be re-hashed from scratch. This is why the operation is $O(n)$ rather than "just copy the array."

When to resize

Three common triggers:

Trigger	When	Typical threshold
Load factor	$\alpha \geq t$	0.5–0.75 (open addr); 1.0+ (chaining)
Chain length (chaining)	longest chain $\geq k$	4–8
Probe count (open addr)	probes $\geq k$	$N/3$ or similar

Load factor is the standard trigger because it's cheap to check (just compare `n_ / m` to the threshold after each insert). The other triggers are more responsive to pathological cases — a few really-bad buckets can justify resizing even if α looks okay.

Growth factor: why doubling?

Suppose you grow by adding a fixed amount ($m' = m + c$) each time. Inserting n items triggers n/c resizes, each $O(n)$ on average, so total work is $O(n^2/c) = O(n^2)$. **Back to amortized $O(n)$ per insert on average** — a disaster.

With $m' = 2m$ (doubling), you resize $O(\log n)$ times over the life of the table. Each resize i rehashes roughly 2^i items. Total rehash work: $1 + 2 + 4 + \ldots + n \approx 2n = O(n)$. Amortized $O(1)$ per insert.

Key idea Same amortization as `std::vector`

This is the same geometric-growth argument that makes `std::vector::push_back` amortized $O(1)$. The rule generalizes: any data structure that occasionally needs to rebuild itself should grow by a constant *factor*, not a constant *amount*. Array lists, hash tables, deques, even JIT code caches — all grow geometrically for this reason.

Why prime sizes?

If table size m has small factors, keys that share those factors will cluster. Example: $m = 10$ and keys are even. Every even key hashes to an even bucket; half the table is wasted.

Prime sizes avoid this: $\gcd(\text{key}, m) = 1$ for most keys, so the modulo spreads them uniformly regardless of key structure. Doubling to the *next prime* ($m = 11 \to 23 \to 47 \to 97$) is a common implementation choice.

Gotcha Resize cost is not invisible

A single insertion that triggers a resize is $O(n)$ — the amortized analysis doesn't help the caller who just hit the spike. For real-time code (games, audio, trading systems), this is a problem: one unlucky

insert blows the frame budget. Fixes include: pre-size the table with `reserve()`, use incremental rehashing (move a few items per operation over time instead of all at once — Redis does this), or accept higher memory and use a never-resize table.

Shrink?

Most implementations *never* shrink automatically. Why:

- Shrinking and regrowing in a tight loop of inserts and removes would thrash.
- The memory "wasted" by a half-empty table is usually cheap compared to the cost of a resize.
- If the user really wants to reclaim memory, they can call `rehash(n)` or move into a fresh table explicitly.

`std::unordered_map` has `rehash(n)` and `reserve(n)` but no auto-shrink. `std::vector` has `shrink_to_fit()` but it's still a manual request.

Aside Load factor thresholds in the wild

`std::unordered_map` default max: 1.0. Java `HashMap`: 0.75. Python `dict`: 2/3 (0.67). Google Abseil Swiss tables: $7/8$ (0.875). Rust `HashMap`: 7/8. Chaining tables can tolerate $\alpha > 1$ because longer chains only hurt linearly; open-addressed tables can't, so they resize earlier. The precise threshold is an engineering knob with tradeoffs on both sides (too low: wasted memory; too high: slow probes).

Key idea Resizing is what makes "$O(1)$ average" work

Without resizing, a hash table's performance degrades linearly as you insert more items. The $O(1)$ guarantee only holds because the implementation keeps α bounded by rebuilding. Resizing is the *mechanism* that preserves the asymptotic guarantee — without it, the analysis breaks down. When you profile a hash table and see occasional slow operations, it's almost always a resize in progress.

7 5.7 Common Hash Functions

Every hash table so far has assumed a hash function that "maps keys to buckets uniformly." This section finally makes that concrete: what does a good hash function *look like*, and what can go wrong?

Definition — Good hash function

A function $h : \text{Keys} \to [0, m)$ that (1) is cheap to compute, (2) distributes keys uniformly across buckets, and (3) uses *all* the bits of the key so that small differences between keys produce different hashes. A *perfect* hash function maps each expected key to a unique bucket with no collisions — achievable only when the full key set is known up front.

Modulo: the simplest, surprisingly subtle

$h(k) = k \bmod m$ is the textbook default.

Example Modulo hash

```
size_t hashModulo(int key, size_t m) {
    return key % m;
}
```

When does it fail? When the keys share a factor with m. If keys are always even and m is even, odd buckets are never used. If $m = 10$ and keys are `10, 20, 30, 40`, every one hashes to bucket 0. This is exactly why prime table sizes are recommended: $\gcd(\text{key}, m)$ is almost always 1, so the modulo spreads the keys out regardless of structure.

Gotcha Powers of 2 and modulo

$k \bmod 2^n$ reduces to k `& ((1 « n) - 1)` — just the low n bits. That's fast, but it throws away all the high-order information. If your keys are clustered in their low bits (pointers on 64-bit systems often are), collisions explode. Either use prime m (accept slower modulo) or mix the bits first (multiply-and-shift, XOR fold).

Mid-square

Square the key, extract the middle bits. The idea: squaring mixes every bit of the key into the middle of the result, so the middle bits depend on the entire key.

Example Mid-square hash (binary)

```
size_t hashMidSquare(uint32_t key, size_t m) {
    uint64_t sq   = uint64_t(key) * key;
    size_t   r    = 10;                    // bits to extract
    size_t   drop = (64 - r) / 2;
    uint64_t mid  = (sq >> drop) & ((1ULL << r) - 1);
    return mid % m;
}
```

Mid-square is a historical curiosity now — multiplicative hashing (below) dominates for integer keys — but it's a useful illustration of the general technique: *mix the bits, then take a slice.*

Multiplicative hashing for strings

For string keys, you can't just take a single character and modulo. You need a function that combines every character into a single hash.

Example djb2, Bernstein's classic

```
size_t hashString(const std::string& s) {
    size_t h = 5381;
    for (char c : s) h = h * 33 + static_cast<unsigned char>(c);
    return h;
}
```

The idea: iteratively fold each character into a running hash by multiplying and adding. The multiplier (33) and seed (5381) are chosen so that small changes in the input produce large changes in the hash — an informal *avalanche* property.

Variants:

- djb2 XOR: `h = h * 33 ^ c` — marginally better distribution
- sdbm: `h = c + (h « 6) + (h « 16) - h` — good for text
- FNV-1a: iterative XOR-then-multiply — widely used
- CityHash, FarmHash, xxHash: modern, SIMD-accelerated, extremely fast

What makes a hash function "good"?

Property	Meaning
Determinism	Same input ⇒ same output, always
Uniform distribution	Each bucket receives roughly n/m keys
Avalanche	Flipping one input bit flips about half the output bits
Cheap	A single arithmetic operation plus modulo, ideally
Non-trivial	Doesn't fall into obvious traps (e.g., identity function)

"Cryptographic strength" is usually *not* required. SHA-256 is a good hash function for ensuring data integrity, but terrible for hash tables (thousands of cycles per call). Hash table hash functions need speed over security.

Gotcha Don't invent your own

Writing a good hash function is harder than it looks. Subtle distribution biases only show up under specific workloads and can be hard to detect. For production code, use `std::hash<T>` (C++), Google's CityHash, or xxHash — all battle-tested. Hand-rolling "just use modulo" is fine for homework; do not ship it in real software handling adversarial input.

C++: `std::hash`

C++ provides `std::hash<T>` for the standard types: ints, pointers, strings, and enums. You can specialize it for your own types by defining `std::hash<YourType>` in the `std` namespace, or supply a custom hasher as a template parameter to the unordered container.

Example Custom hasher

```
struct Point { int x, y; };

struct PointHash {
    size_t operator()(const Point& p) const noexcept {
        // combine two hashes --- the standard "boost::hash_combine"
            pattern
        size_t h1 = std::hash<int>{}(p.x);
```

```
        size_t h2 = std::hash<int>{}(p.y);
        return h1 ^ (h2 + 0x9e3779b9 + (h1 << 6) + (h1 >> 2));
    }
};

std::unordered_set<Point, PointHash> points;
```

The magic number `0x9e3779b9` is $\frac{2^{32}}{\phi}$ (ϕ = golden ratio) — a widely-used bit-mixing constant that gives good distribution across field combinations. Boost's `hash_combine` uses this; many custom hashers copy the idiom.

Aside Hash randomization: security vs. reproducibility

Some languages randomize their hash seeds at startup to defend against hash-collision DoS attacks (Python 3.3+, Ruby, Rust). A side effect: iteration order of a hash map is now *nondeterministic* between program runs. Code that depends on consistent iteration order breaks subtly. C++'s standard doesn't mandate randomization, though some implementations offer it as a build option.

Key idea The big picture

A hash table's speed is only as good as its hash function. Any of the collision-resolution schemes (chaining, linear, quadratic, double) will perform terribly if the hash function is poor — because they all assume approximately-uniform distribution. When your hash table is slow, suspect the hash function first, the load factor second, and the collision strategy last.

Universal hashing: the principled defense against adversarial input

The hash randomization notebox above is a practical sketch; the underlying theory is *universal hashing*, due to Carter and Wegman (1979). It is the standard answer to "what if my hash function's worst-case input shows up?" — a question every fixed hash function leaves dangling.

Definition — Universal hash family

A finite collection $\mathcal{H}$ of hash functions $h : U \to \{0, 1, \ldots, m-1\}$ is **universal** if for every pair of distinct keys $k, l \in U$,

$$\Pr_{h \in \mathcal{H}}\big[h(k) = h(l)\big] \;\le\; \frac{1}{m}.$$

The probability is over the random choice of h, *not* over the keys.

The motivation is the gap between average-case and worst-case for *any single* hash function. Pick any fixed h and an adversary can find n keys that all hash to the same bucket, giving $\Theta(n)$ chains. Universal hashing escapes this by randomizing the function itself at the start of execution: the adversary picks the keys, but the program picks the hash function *independently and after* the keys are chosen, so worst-case inputs no longer exist for any particular run.

Example The $h_{a,b}$ family from CLRS Ch 11.3.3

```
// Universal hash family h_{a,b}(k) = ((a*k + b) mod p) mod m
// where p > u (universe size) is a fixed prime, and a, b are
// chosen at random at startup with a in [1, p-1], b in [0, p-1].

struct UniversalHash {
    uint64_t a, b;
    uint64_t p;            // prime > universe size
    size_t   m;            // table size

    UniversalHash(size_t table_size, uint64_t prime) : p(prime), m(
        table_size) {
        std::mt19937_64 rng(std::random_device{}());
        a = 1 + rng() % (p - 1);   // a in [1, p-1], nonzero
        b =     rng() %  p;        // b in [0, p-1]
    }

    size_t operator()(uint64_t k) const {
        return ((a * k + b) % p) % m;
    }
};
```

Aside Why universal hashing implies short chains

The expected chain length at $h(k)$ is now bounded by $1 + \alpha$ *regardless of the keys.* Sketch (CLRS Theorem 11.3, OCW lec4): for each pair k, l of distinct keys, define indicator $X_{kl} = \mathbf{1}[h(k) = h(l)]$. By the universality condition, $E[X_{kl}] = \Pr[h(k) = h(l)] \leq 1/m$. The number of keys colliding with k is $Y_k = \sum_{l \neq k} X_{kl}$, so by linearity of expectation,

$$E[Y_k] \;=\; \sum_{l \neq k} E[X_{kl}] \;\leq\; \sum_{l \neq k} \tfrac{1}{m} \;\leq\; \tfrac{n-1}{m} \;\leq\; \alpha.$$

The expected length of the bucket containing k is $E[Y_k] + 1 \leq 1 + \alpha$. *The same bound as simple uniform hashing, but now it follows from the algorithm's randomness, not from an assumption about the input.*

Key idea How real systems use this

Few production hash tables ship a full universal-family implementation; the constant factors don't justify it for typical workloads. What real systems *do* ship is the same idea in a weaker form: a process-startup random seed combined with a fast non-cryptographic hash. Python's `hash` (since 3.3), Rust's `RandomState`, Java's salted `String.hashCode` on recent JVMs — all of them randomize at process start so an attacker cannot precompute colliding inputs. The formal universal-family construction is the textbook answer; salted-hash randomization is the production answer. The defense is the same in spirit: don't let the adversary pick the function.

8 5.8 Direct Hashing

Direct hashing is the degenerate case: use the key itself as the bucket index. No modulo, no mixing, no collisions. Whenever it's feasible, it's the fastest possible hash table. Unfortunately, it's rarely feasible.

Definition — Direct access table

A hash table in which the hash function is the identity: $h(k) = k$. The bucket for key k is always table$[k]$. No collisions can occur because every key has its own bucket.

When it works

Direct hashing requires *every* potential key to have a reserved slot. If your keys are integers in $[0, K)$, you need a table of size K.

Example Direct-access table

```
// Suitable only when keys are small non-negative integers.
constexpr size_t K = 10000;
std::optional<Item> table[K];

void insert(const Item& it)   { table[it.key] = it; }
void remove(int key)          { table[key].reset(); }
Item* find(int key) {
    return table[key] ? &*table[key] : nullptr;
}
```

Three lines per operation, all $O(1)$ *worst case* — not average. No hash function to compute, no collision handling, no resizing.

The two killer limitations

Limit	Consequence
Keys must be small non-negative integers	Can't directly handle strings, negative numbers, or large key spaces
Table size = max key + 1	A key space of 2^{32} needs a 4-billion-slot array (16 GB if each slot is 4 bytes, more if you store anything bigger)

If the key space is dense — say, student IDs 0–9999 and you have 8000 active students — direct hashing is perfect: a 10,000-slot array, 80% utilized, all operations $O(1)$ with zero overhead.

If the key space is sparse — say, US social security numbers (9 digits, 10^9 possible values) but your table has a few thousand people — direct hashing wastes 99.9999% of the array. You need a regular hash table.

Key idea Direct hashing is the ceiling

Direct hashing represents the best a hash table can do: one memory access per operation, zero collision work. Every "real" hash table is an approximation of this under memory pressure. When you profile a hash-heavy workload and it's still slow, the question to ask is "how close am I to direct hashing behavior?" — the answer tells you whether the slowness is unavoidable or an artifact of your chosen scheme.

Where direct hashing actually shows up

Don't dismiss it as purely theoretical. Small-keyspace direct access tables are genuinely common:

- **ASCII character lookup tables** (256 slots) — tokenizer tables, character-class tests (`isalnum`, `isspace`)
- **Byte-to-hex** conversion (256 slots holding two-character strings)
- **Color palettes** in graphics (indexed color: the "hash" is just the palette index)
- **File descriptor tables** in OS kernels (fd is already an array index)
- **CPU register mapping** in compilers (register number → allocation info)
- **Symbol tables in assemblers** when symbol IDs are sequential

Every one of these uses direct indexing by design because the key space is small and dense.

Gotcha Negative keys need translation

If your keys include negative integers, you can still use direct hashing — just add an offset. Keys in $[-100, 100]$ map to indices $[0, 200]$ via `index = key + 100`. This extends direct hashing to any *bounded* integer range.

Comparison

Strategy	Worst case	Collisions?	Space
Direct hashing	$O(1)$	None	$O(K)$ (key range)
Chaining	$O(n)$	Handled	$O(n+m)$
Open addressing	$O(n)$	Handled	$O(m)$

Direct hashing trades space for time: you pay for a slot for every possible key, but you get the best possible access time. Regular hash tables reverse the deal: save memory, accept occasional collision handling.

Aside Perfect hashing: the middle ground

For static key sets (known up front, never changes), you can construct a *perfect hash function* — one that maps each key to a unique bucket in a tightly-sized table. No wasted space, no collisions. Classic use: compiler keyword tables (the set {`if`, `else`, `while`, `for`, ...} is fixed; you can hand-tune a hash function that maps each to a distinct small integer). Tools like `gperf` generate perfect hash functions from a word list. It's direct hashing for the modern world — limited to static data but optimal when applicable.

Key idea Why this section exists

Direct hashing is deliberately a *counterexample* — it shows what you'd do if memory were free. Every technique in sections 5.1–5.7 exists because direct hashing's memory cost is prohibitive. The moment you squint at a hash table problem and realize "oh, my keys are small integers," you should stop and consider an array.

9 5.9 Hashing for Cryptography

Hashing has a second life outside data structures: as a *cryptographic primitive*. Cryptographic hash functions share the "deterministic function from input to fixed-size output" shape with hash-table functions, but their design goals — and their cost — are radically different.

Definition — Cryptographic hash function

A hash function with three additional properties: (1) *pre-image resistance*: given a hash value h, it is computationally infeasible to find any input x such that $\text{hash}(x) = h$; (2) *second pre-image resistance*: given an input x_1, it is infeasible to find a different x_2 with the same hash; (3) *collision resistance*: it is infeasible to find any two inputs with the same hash.

What "infeasible" means

A good cryptographic hash with an n-bit output requires approximately 2^n work to find a pre-image and $2^{n/2}$ work to find a collision (the birthday bound). For $n = 256$, that is $\approx 10^{77}$ operations — more than the age of the universe on all the computers ever built.

Compare this to a hash-table hash function like djb2. djb2 is fast (a few nanoseconds per byte) but trivially easy to reverse or collide on demand. It's not secure; it was never designed to be.

Goal	Hash-table hash	Cryptographic hash
Speed	≤ 1 ns/byte	1–10 ns/byte (SHA-256)
Output size	32–64 bits	128–512 bits
Reversibility	Often easy	Infeasible
Collisions	Acceptable (handled)	Infeasible to produce
Use cases	Lookups	Integrity, signatures, passwords

Data integrity

Download a file plus its hash. Recompute the hash locally. If they match, the file wasn't corrupted or tampered with.

Example Verifying a download with SHA-256

```
$ sha256sum ubuntu-24.04.iso
c37f6b0e...  ubuntu-24.04.iso

# Compare against the published hash on ubuntu.com.
# Match   -> bits are correct
# Mismatch -> file is corrupt or modified
```

Historical timeline of algorithms:

- **MD5** (1991): 128-bit output. Collision attacks demonstrated in 2004; **broken**, do not use for security.
- **SHA-1** (1995): 160-bit output. Collision demonstrated in 2017 (SHAttered); **deprecated**.
- **SHA-256 / SHA-3**: current standards. 256–512 bit outputs, no known attacks faster than brute force.

Gotcha Never use MD5 or SHA-1 for security

Both have been publicly broken. They remain fine for *non-security* use (checksums, content-addressable storage keys) where an attacker isn't trying to cause collisions. But for anything where deception is a threat model — signatures, certificates, password hashing — use SHA-256 or better.

Password hashing

When a user signs up, don't store their password. Store `hash(password)`. At login, hash the submitted password and compare. If the database is stolen, attackers have hashes, not passwords — and with a good hash, inverting them is infeasible.

Example Naive password hashing (don't do this)

```
stored = sha256(password)   // bad
```

Two attacks on naive password hashing:

1. **Rainbow tables**: attackers precompute hashes for billions of common passwords. If your hash is in the table, password recovered instantly.
2. **Bulk cracking**: modern GPUs compute billions of SHA-256 hashes per second. For short or weak passwords, brute force is tractable.

The two standard defenses:

Defense	What it does
Salt	Append a random per-user value before hashing. Forces the attacker to recompute the rainbow table for every user — no more precomputation.
Slow hash	Use a deliberately expensive function (bcrypt, scrypt, Argon2) that takes 100ms–1s per hash. Kills bulk cracking: a million-password attack now takes 12 CPU-days instead of a second.

Example Correct password hashing

```
salt   = random_bytes(16)                  // unique per user
stored = bcrypt(password + salt, cost=12)  // slow
// store (salt, stored) in the database
```

`bcrypt`, `scrypt`, and `Argon2` are the current standards. Argon2 won the Password Hashing Competition in 2015 and is the recommended default for new systems.

Key idea The design tension: fast vs. slow

Cryptographic hashes usually want to be *fast* (you hash a lot of data for integrity). Password hashes want to be *slow* (you hash once per login, and slow is the point). This is why "use SHA-256 for

passwords" is wrong — SHA-256 was engineered for speed, the opposite of what password hashing needs. Always use a dedicated password hash for passwords.

Where else does cryptographic hashing show up?

- **Digital signatures**: sign the hash of the document, not the document itself. Cheaper, and works for any size.
- **Blockchain / content addressing**: each block references the hash of the previous block. Git does the same with commits.
- **Merkle trees**: hash-based structures that let you verify membership in a large set without downloading the whole set.
- **HMAC**: keyed hash functions used to authenticate messages.
- **Commitment schemes**: hash now, reveal later — used in auctions, coin-flipping protocols, etc.

Aside Cryptographic hashes and hash tables in one program

A modern web server might use both. SHA-256 to verify TLS certificates and check uploaded file integrity; a fast non- cryptographic hash (like xxHash or std::hash) to back `unordered_maps` in the session cache. The two systems have different threat models and different cost profiles; it's normal for one program to use both.

Key idea Chapter 5 in retrospect

You started with a simple idea: *compute the address instead of searching for it.* Then spent eight sections on the engineering reality: hash functions that spread keys out, collision resolution strategies, resizing to bound load, when direct hashing applies. This final section pivots to a completely different use of the same mathematical tool. Cryptographic hashing *looks* like data-structure hashing but has almost nothing else in common — the only shared idea is "deterministic function, fixed-size output." The design space between a 4-nanosecond hash-table hash and a 100-millisecond Argon2 password hash is enormous, and the choice depends entirely on what you're defending against.

What this connects to

Aside Forward links to later chapters

- **Chapter 6 (BSTs).** BSTs give you a sorted alternative to hash tables. Choose BST (ordered set, range queries) vs. hash (fastest point lookup) – the tradeoff is on the exam.
- **Graph algorithms** (opt. ch. 10). "Visited" sets in BFS / DFS are hash sets; Dijkstra's "distance" lookup is a hash map; hashing keeps graph traversal at expected $O(V + E)$.
- **Dynamic programming memoization.** Top-down DP stores already-computed subproblems in a hash map. The efficiency of DP in the average case is a hash-table claim.
- **C++ production idiom.** `std::unordered_map<K, V>` is chaining; `std::map<K, V>` is a red-black tree (ch. 6 territory). Knowing which to pick is a real decision on every CS 300 assignment from here on.

- **Linked lists** from ch. 4 are what chaining buckets *are*. The node + pointer drills from ch. 4 pay off the moment you trace a chaining collision.

Companion materials. One-page cheat sheet: `cheat_sheets/ch_5.tex`. Practice prompts (problem → pseudocode → C++): `practice_prompts/ch_5.md`.